

APPLICATION DEVELOPMENT - I

Attendance Tracker system REPORT

Submitted by

GOKULNATH D - 7178241215

HARITHA A 7178241218

In partial fulfillment for the award of the degree

Of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIALINTELLIGENCE AND DATA SCIENCE



ANNA UNIVERSITY, CHENNAI - 600 025

April 2025



CERTIFICATE

This is to certify that the **Application Development ATTENDANCE TRACKER SYSTEM Report** submitted by **GOKULNATH D (7178241215) , HARITHA A (7178241218)** is work done by him and submitted during **2025 -2026** academic year, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Artificial Intelligence And Data Science**.

Faculty guide

Head of the Department

TABLE OF CONTENTS

CHAPTER NO	TOPICS	PAGE NO
	ACKNOWLEDGEMENT	I
	ABSTRACT	Ii
	LIST OF FIGURES	Iii
	LIST OF ABBRIVATIONS	Iv
1	INTRODUCTION TO FRONT-END DEVELOPMENT	1
	1.1 Key Components Of front-end Development	1
	1.2 Frameworks and Libraries	2
	1.2.1 React.js	2
	1.2.2 Angular	3
	1.2.3 Vue.js	3
	1.2.4 jQuery	3
	1.2.2 Bootstrap	3
	1.3 Version Control	3
	1.4 Responsive Design	
	1.5 APIs	3
	1.6 Web Performance Optimization	3
	1.7 Browser developer Tools	3
	1.8 The web development process	
	1.9 Current trends in web development	
	1.9.1 JAMstack	
	1.9.2 API-First Design	
	1.9.3 SSR with Front-End Libraries	

2 INTRO TOOLS AND TECHNOLOGIES USED

2.1 Introduction to tools

2.2 Local Storage

- 2.2.1 Purpose of using local storage
- 2.2.2 How data is stored and retrieved
- 2.2.3 Advantages
- 2.2.4 Limitations

2.3 VS Code

- 2.3.1 Features
- 2.3.2 Extensions
- 2.3.3 Benefits of using VS code
- 2.3.4 Debugging Techniques

3 HTML

- 3.1 What is HTML
- 3.2 Purpose of using
- 3.3 Key elements used
 - 3.3.1 Eample code for key elements used
- 3.4 Structure of attendance page
- 3.5 sample code

4 CSS

- 4.1 What is css**
- 4.2 Types of CSS**
- 4.2 Role of CSS
- 4.3 Internl vs external styling
- 4.4 CSS properties used
- 4.5 Designing responsive layouts
- 5.6 sample code

5 JAVASCRIPT

- 5.1 What is JS
- 5.2 Functionality added
- 5.3 Form validation Techniques
- 5.4 Attendance Status update logic
- 5.5 Elements of js
- 5.6 Sample co

6 IMPLEMENTATION

- 6.1 Login page**
- 6.2 Home page**
- 6.3 Students page**
- 6.4 Subject page**
- 6.5 Report page**
- 6.6 Attendance page**

7 CONCLUSION

REFERENCE

ACKNOWLEDGEMENT

I extend my gratitude to my industrial mentor, academic guide, and the faculty of the Department of Artificial Intelligence and Data Science, **Anna University**, for their encouragement and guidance.

It is a moment of immense pride for me to reveal my profound thanks to our respected Principal, **Dr. V. Kumar Chinnaiyan M.E, Ph.D.** who happens to be a striving force in all our endeavors.

I express my sincere thanks to our **HoD Name**
Head of Science and Humanities .

I express my sincere thanks to my Mentors **Guide Name**
who provided me with the necessary guidance and encouragement.

Without the help and support of all the mentioned people, I may not be able to fulfill my internship and learn different things. I would like to thank all those people for their valuable contribution and proper guidance throughout my internship period.

Finally, I must acknowledge with due respect the constant support and patience of my parents.

ABSTRACT

The Attendance Tracker System is a web-based application developed to simplify and automate the management of student attendance. Traditional manual attendance recording methods are often time-consuming and prone to errors. This system addresses these problems by offering a digital platform where attendance can be recorded, managed, and reported accurately through an organized website structure.

The system is designed with multiple key pages: a Home Page to navigate through the site, a Students Page to add and manage student information, a Subjects Page to organize academic subjects, a Report Page to view attendance summaries, and a Generate Page to produce detailed attendance reports. Each page is designed with user-friendliness in mind to ensure smooth interaction and easy access to data.

The Students Page allows users to add, edit, and delete student records securely, while preventing duplicate entries by matching student names and IDs. The Subjects Page manages the subjects under which attendance will be tracked. Once students and subjects are entered, the Generate Page enables users to mark attendance for specific classes and sessions. The Report Page compiles all attendance records into clear, easy-to-understand reports.

The technology stack used for this project includes web development tools like HTML, CSS, and JavaScript for the frontend. The website structure ensures that all operations, from data input to report generation, are simple, organized, and reliable. It also lays a strong foundation for future upgrades, such as linking with databases or incorporating login systems for enhanced security.

In conclusion, the web-based Attendance Tracker System improves the accuracy and efficiency of attendance management by replacing traditional manual methods. Its simple yet effective multi-page design ensures that administrators and users can handle attendance tasks smoothly. This project demonstrates the potential of small web applications to solve real-world management problems with speed and precision.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
1.1	Introduction to web development components	1
1.2	Basic HTML page	6
3.1	Output of css	16
4.1	Output of Javascript	21
5.1	Object view in portal	51
5.2	Brand and Model	51
5.3	Select model in site	52

LIST OF ABBRIVATIONS

- **AJAX** - Asynchronous JavaScript and XML
- **API** - Application Programming Interface
- **CDN** - Content Delivery Network
- **CMS** - Content Management System
- **CSS** - Cascading Style Sheets
- **DOM** - Document Object Model
- **FTP** - File Transfer Protocol
- **GIF** - Graphics Interchange Format
- **HTML** - Hypertext Markup Language
- **HTTP** - Hypertext Transfer Protocol
- **HTTPS** - Hypertext Transfer Protocol Secure
- **IP** - Internet Protocol
- **JSON** - JavaScript Object Notation
- **JS** - JavaScript
- **MVC** - Model-View-Controller
- **PHP** - Hypertext Preprocessor
- **REST** - Representational State Transfer
- **SEO** - Search Engine Optimization
- **SQL** - Structured Query Language
- **URL** - Uniform Resource Locator

CHAPTER 1

INTRODUCTION TO FRONT-END DEVELOPMENT

1.1 Key components of front-End development

1.1.1 HTML (Hyper Text Markup Language):

- HTML (HyperText Markup Language) is the standard language used to create and structure web pages
- . It consists of a series of elements, represented by tags, that define the content and layout of a webpage
- HTML allows you to add text, images, links, videos, and forms to a page, making it the basic building block of all websites.
- By using simple tags like `<h1>` for headings, `<p>` for paragraphs, and `<a>` for links, HTML helps organize and display content on the web

1.1.2 CSS (Cascading Style Sheets):

- CSS (Cascading Style Sheets) is a language used to style and format the layout of web pages.
- While HTML defines the structure of a webpage, CSS is responsible for how it looks.
- CSS allows you to control the appearance of elements, such as changing colors, fonts, sizes, and spacing, as well as arranging content in different layouts.
- You can apply styles to a single page or across multiple pages using selectors, making it a powerful tool for creating attractive and responsive web designs.

1.1.3 JavaScript (JS):

- JavaScript is a programming language that adds interactivity and functionality to web pages.
- While HTML structures the content and CSS styles it, JavaScript allows you to make the page interactive by responding to user actions like clicks, typing, or mouse movements.
- It can be used to create dynamic features such as form validation, animations, and interactive maps. JavaScript runs in the browser, making it an essential tool for modern web development, helping you create engaging and responsive websites.

1.2 Frameworks and Libraries:

1.2.1 React.js:

Makes building interactive websites easier by allowing developers to create small, reusable pieces of code called components.

1.2.2 Angular:

A full framework that gives developers everything they need to build complex web apps, like handling data, user interactions, and more.

1.2.3 Vue.js:

A lightweight and flexible tool for building modern websites, focusing on making it easy to integrate with other projects or scale up for larger apps.

1.2.4 jQuery:

A simpler tool for handling things like changing content on a webpage or responding to user clicks. It's older but still useful for small projects, though less commonly used with newer frameworks.

1.2.2 Bootstrap:

A CSS framework that provides ready-made styles and layouts to help create responsive websites quickly. It makes sure your site looks good on phones, tablets, and desktops.

1.3 Version Control

Version control helps developers keep track of changes made to their code and work together more effectively.

- **Git:** A tool that tracks changes to your code over time, allowing you to go back to previous versions if needed. It also makes it easier to collaborate with others without overwriting each other's work.
- **Git repositories:** A place where your code is stored, often on platforms like GitHub or GitLab, which also make it easier to share and work with a team. These platforms help organize project versions and manage team contributions.

1.4 Responsive Design

Responsive design ensures that websites look great and are easy to use on all devices, from large desktop monitors to small mobile screens.

The approach of designing a website so that it automatically adjusts to different screen sizes, ensuring a good user experience no matter the device.

CSS Media Queries: Special CSS rules that apply only when certain conditions are met (like the screen width or device type), allowing you to adjust the layout of your website for various devices.

1.5 APIs

APIs allow different software systems to talk to each other, enabling the front-end (what users see) to communicate with the back-end (the server and database).

APIs enable the front-end of a website or app to interact with the back-end services, such as sending data (like a form submission) or retrieving information (like user data or products).

APIs are often used to fetch data from the back-end (e.g., user profiles, product lists) and display it dynamically on the front-end without needing to reload the page. This helps create interactive and seamless user experiences.

1.6 Web Performance Optimization:

Web performance optimization focuses on making websites load faster and run smoothly for better user experience.

- **Techniques:** This includes practices like compressing images to reduce file sizes, lazy loading (loading images and content only when needed), and code splitting (breaking up code into smaller pieces to load only what's necessary).
- **Focus:** The goal is to minimize load times and provide smooth interactions, ensuring the site performs well across different devices and internet speeds.

1.7 Browser developer Tools

These are built-in tools in web browsers (e.g., Chrome DevTools) that help developers inspect web pages, debug code, and test performance. These tools allow you to check the HTML, CSS, and JavaScript of a webpage, identify issues, debug problems, and optimize the front-end to improve performance.

Key Feature:

- Inspect elements
- Console for logging
- Network monitoring (XHR, fetch)
- JavaScript debugging and profiling
- Performance analysis
- Mobile emulation

1.8 Web development process

The web development process typically involves several stages to create a fully functional and user-friendly website or web application. Here's a breakdown of the typical steps involved:

- Planning and Research:
- Design:
- Development:
- Testing:
- Deployment:
- . Maintenance:
- Optimization (Ongoing)

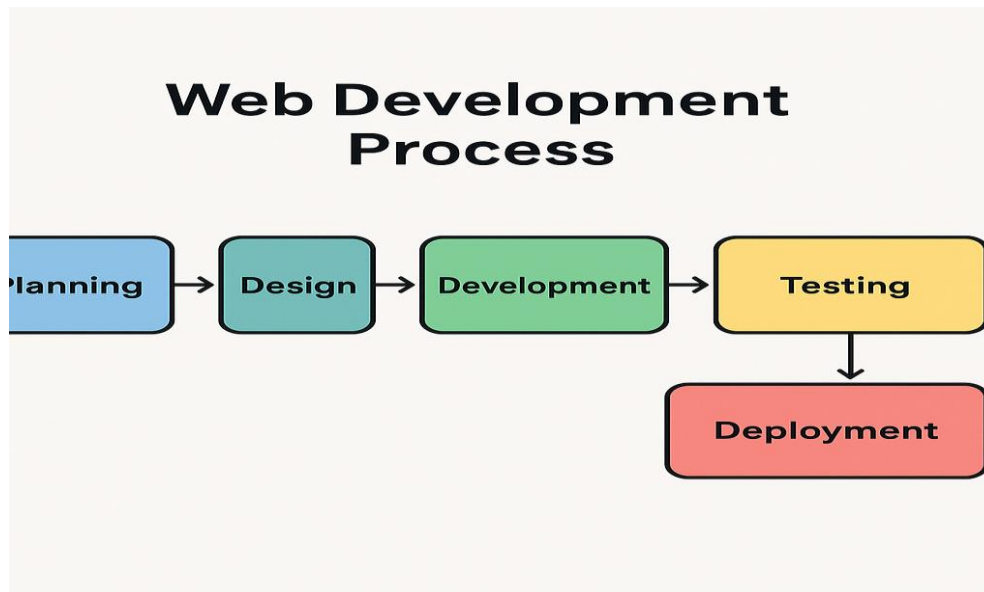


Figure 1.1: Web Development process

1.9 Current trends in web development

1.9.1 JAMstack (JavaScript, APIs, and Markup)

- **What is JAMstack?**
JAMstack is a modern web architecture built around client-side JavaScript, reusable APIs, and static content. It emphasizes speed, security, and scalability.
- **Relevance to Your Project:**
Your Attendance Tracker could benefit from JAMstack, especially for serving static content like attendance reports, while using APIs to fetch dynamic data like student records.

1.9.2 API-First Design

- **What is API-First Design?**
In API-first design, you build the APIs first before the front-end and back-end applications, ensuring flexibility and ease of integration.
- **Relevance to Your Project:**
For future scalability, you can design APIs first to manage attendance data. This will allow easier integration with other systems or even future mobile applications.

1.9.3 Server-Side Rendering (SSR) with Front-End Libraries

- **What is it?**
SSR involves rendering your web pages on the server side before sending them to the client, improving performance and SEO.
- **Relevance to Your Project:**
For your Attendance Tracker, using SSR with a front-end framework like React or Vue.js can help reduce page load times and improve the SEO of your site, ensuring that important pages like reports or student lists are easily indexable by search engines.

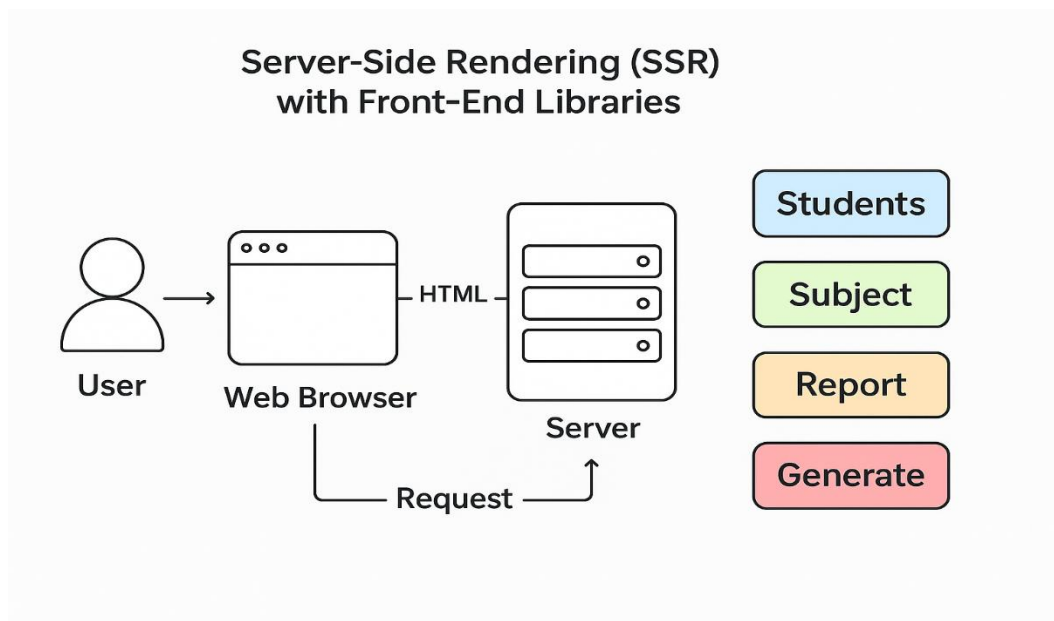


Figure 1.2: SSR with Front- end librries

CHAPTER 2

TOOLS AND TECHNOLOGIES USED

2.1 Introduction to tools

In the development of the Attendance Tracker System, several essential tools and technologies were utilized to ensure an efficient, user-friendly, and reliable application. Front-end development was primarily carried out using HTML, CSS, and JavaScript, providing a structured layout, attractive design, and interactive features. These technologies helped in building responsive web pages that allow users to easily manage student attendance. Additional libraries and frameworks, if any, further enhanced the functionality and appearance of the system. Each tool played a crucial role in creating a seamless experience for both administrators and users.



Figure 2.1 :Tools used in the attendance tracker system

2.2 Local Storage

2.2.1 Purpose of Using Local Storage

- Local Storage is used in the Attendance Tracker project to **save student data and**

attendance status directly in the user's browser.

- It allows the project to **retain information** even if the page is refreshed or the browser is closed and reopened.
- Without Local Storage, all the entered data would be lost when the user accidentally reloads the page, causing inconvenience.
- By using Local Storage, students' attendance records are preserved until the user manually clears them or uses a "Clear All" button.
- It provides a simple way to make the Attendance Tracker behave like a basic offline web application without needing a server or database.

2.2.2 How Data is Stored and Retrieved

Data in Local Storage is stored as **key-value pairs**, and both the key and value must be strings.

In the Attendance Tracker:

- When a student is added, JavaScript **serializes** the student details (Name, ID, Attendance Status) into a JSON string and **saves it** under a specific key.
- When the page loads, JavaScript **retrieves** the stored JSON string using the key, **parses** it back into a JavaScript object/array, and **displays** it in the attendance table. The main functions used are:
 - `localStorage.setItem(key, value)` — To store data.
 - `localStorage.getItem(key)` — To retrieve data.
 - `localStorage.removeItem(key)` — To delete specific stored data if needed.

2.2.3 Advantages

- **Persistence:** Data stays even after closing or refreshing the browser.
- **Offline Availability:** Attendance Tracker works even without an internet connection.
- **Simple API:** Very easy to use with JavaScript, no special setup needed.
- **Fast Access:** Retrieval and storage happen almost instantly because the data is local.
- **Improves User Experience:** Users don't lose their entered data accidentally.

2.2.4 Limitations

- **Limited Storage Capacity:** Usually about **5MB** only — not suitable for very large data.
- **Security Risk:** Data is stored as plain text and can be viewed/modified through browser developer tools.
- **Single Browser Only:** Data stored in one browser (like Chrome) won't appear in another (like Firefox).
- **No Server Synchronization:** Cannot automatically sync data between devices.

- **Data Overwriting:** Keys must be managed carefully; otherwise, new data can overwrite old data by mistake.

2.3 VS Code

Visual Studio Code (VS Code) is a free, open-source code editor developed by Microsoft. It is lightweight yet powerful, offering built-in support for development operations like debugging, task running, and version control. It supports a wide range of programming languages and frameworks through its rich extension marketplace.

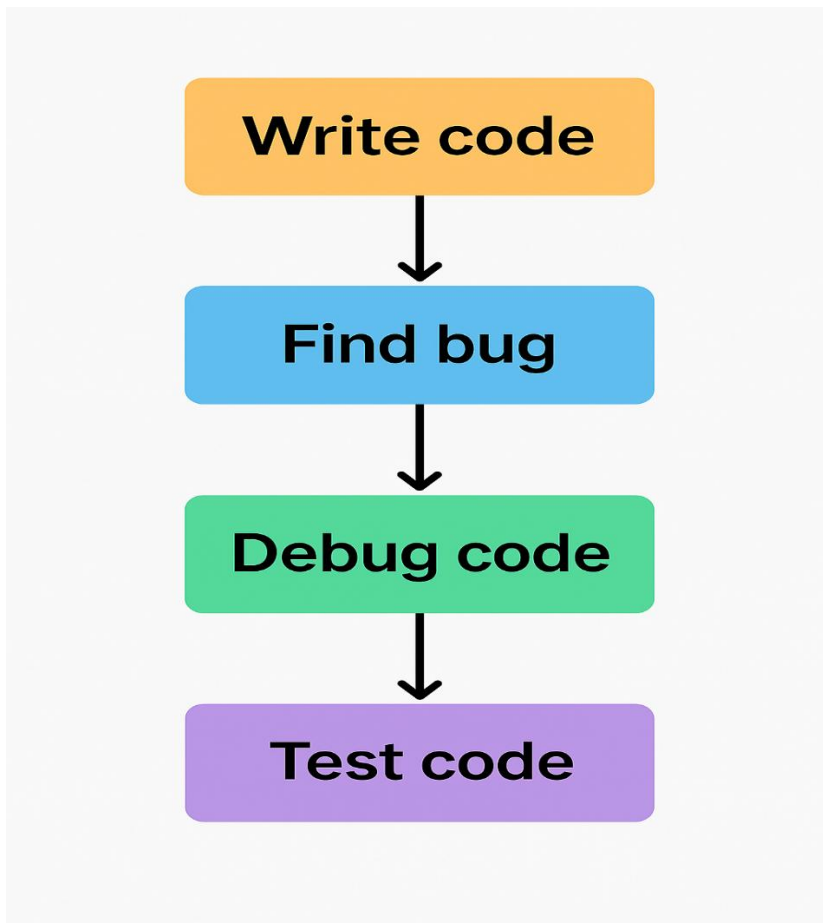


Figure 2.2: steps for using VS code

2.31 Features

- **Intelligent Code Completion:** VS Code offers IntelliSense for smart code completions based on variable types, function definitions, and imported modules.
- **Built-in Git Integration:** Easily manage version control operations directly from the editor.
- **Customizable Themes and Layouts:** Users can modify the look and feel according to their preference.
- **Multi-root Workspaces:** Ability to work with multiple project folders in one window.

- **Integrated Terminal:** Allows you to open a terminal inside the editor, making it easy to run scripts and commands.
- **Snippets and Code Navigation:** Helps developers write code faster and navigate large codebases efficiently.

2.3.2 Extensions

- **Prettier:** For automatic code formatting.
- **Live Server:** Launches a development local server with live reload for static and dynamic pages.
- **Python:** Provides rich support for the Python language, including features like linting and debugging.
- **ESLint:** Helps in identifying and fixing JavaScript code quality issues.
- **C/C++:** Adds debugging, code browsing, and IntelliSense support for C and C++.
- **Docker:** Simplifies container development workflows.

Extensions significantly enhance the capabilities of VS Code to suit different development needs and technologies.

2.3.3 Benefits of using VS Code

- **Lightweight and Fast:** Unlike full-fledged IDEs, VS Code is faster to install and run.
- **Cross-platform Support:** Available on Windows, macOS, and Linux.
- **Highly Customizable:** Supports extensions, settings, and themes, making it adaptable to individual workflows.
- **Strong Community Support:** Active community and frequent updates provide new features and quick bug fixes.
- **Extensive Language Support:** Covers nearly all programming languages through extensions.
- **Seamless Debugging:** Provides an excellent debugging experience right inside the editor.

2.3.4 Debugging Techniques

- **Breakpoints:** Set breakpoints in your code to pause execution at specific points and inspect values.
- **Watch Variables:** Monitor specific variables for changes during code execution.
- **Call Stack Inspection:** View the call stack to trace the sequence of function calls.
- **Step Into/Over/Out:** Control execution flow to dive into functions, skip over them, or step out after inspection.
- **Integrated Debug Console:** Evaluate expressions, execute commands, and inspect outputs while paused at breakpoints.
- **Launch Configurations:** Create custom debug setups using launch.json to define how projects are debugged.

CHAPTER 3

HTML (Hyper Text Markup Language)

3.1 WHAT IS HTML

- **HTML** (HyperText Markup Language) is the standard language used to create web pages and web applications.
- It structures the content on the web, allowing users to display text, images, links, forms, and other multimedia elements.
- HTML is fundamental to web development, as it forms the foundation for creating interactive, content-driven websites.
- HTML is not a programming language; it's a **markup language**. This means it uses tags to define and describe content, but it doesn't have the logic or functionality that a programming language like JavaScript or Python has.
- HTML works by using elements or "tags" that tell the browser how to display and format the content on a webpage.

3.2 Purpose of Using HTML

HTML is the standard markup language used for creating web pages. It serves several important purposes:

- **Structuring Content:** HTML allows for the creation of headings, paragraphs, lists, tables, and forms, which are essential for organizing data in a readable and accessible format.
- **Linking Resources:** It helps link external resources like stylesheets (CSS), scripts (JavaScript), and multimedia files (images, audio, video).
- **Creating Forms:** In an Attendance Tracker System, HTML forms allow users to input, modify, and submit attendance data. Forms can capture student IDs, names, attendance status, and timestamps.
- **Building Navigation:** It provides anchor (<a>) tags that enable users to navigate between different pages, such as the homepage, attendance records, and student details page.

3.3 Key Elements Used

HTML provides a variety of elements used to build a functional and well-structured webpage:

- **<html>:** The root element that defines the entire HTML document.
- **<head> and <body>:** The <head> contains metadata, title, and links to external files like CSS and JS, while the <body> contains the actual content displayed on the page.
- **<header> and <footer>:** These semantic tags define the header and footer of the page, which typically include navigation and contact information, respectively.

- **<table> and <tr>:** For displaying data in a tabular format, such as attendance records. <table> holds the structure, and <tr> defines each row.
- **<form> and <input>:** Forms allow the user to submit data. Inputs like text fields and radio buttons are used to enter or select information.
- **<nav>:** A semantic element used for navigation menus, helping users to jump between different sections of the website.
- **<section> and <article>:** These elements allow you to divide content logically into sections or articles, improving readability and organization.

3.3.1 Example code for key elements used

Code:

```
<!DOCTYPE html>

<head>

  <meta charset="UTF-8">

  <title>Attendance Tracker System</title>

</head>

<body>

  <h1>Attendance Tracker System</h1>

  <form>

    <fieldset>

      <legend>Mark Attendance</legend>

      <label for="studentId">Student ID:</label><br>

      <input type="text" id="studentId" name="studentId" required><br><br>

      <label for="studentName">Student Name:</label><br>

      <input type="text" id="studentName" name="studentName" required><br><br>

      <input type="submit" value="Mark Present">

      <input type="reset" value="Clear">

    </fieldset>

  </form>

  <hr>
```

```
<h2>Attendance List</h2>
```

```
<table border="1">
```

```
<thead>
```

```
<tr>
```

```
<th>Student ID</th>
```

```
<th>Student Name</th>
```

```
<th>Status</th>
```

```
</tr>
```

```
</thead>
```

```
</table>
```

```
</body>
```

```
</html>
```

Attendance Tracker System

Mark Attendance

Student ID:

Student Name:

Please fill out this field.

Mark Present

Clear

Attendance List

Student ID	Student Name	Status
------------	--------------	--------

Figure 3.1 Website designed only using HTML

3.3 Structure of Attendance Pages

The attendance pages in your system will have a clean and organized structure, consisting of several parts:

- **Header:** Contains the site logo, page title, and primary navigation links (e.g., Home, Add Attendance, View Attendance).
- **Main Content:** Displays the attendance data in a user-friendly manner, such as a table with columns for student names, IDs, and attendance status (present/absent).
- **Forms for Data Entry:** These sections allow teachers or admins to input new attendance records. Forms will include fields like student ID, date, and attendance status.

- **Footer:** Includes secondary navigation or contact information, privacy policies, or links to related pages.
- **Interactive Features:** Use of buttons (like Submit or Edit) for adding, editing, and deleting attendance data.

3.4 Importance of Semantic Tags

Semantic tags are HTML elements that clearly describe their meaning in the context of the content they contain. Using semantic tags improves accessibility, search engine optimization (SEO), and code readability.

- **Improved Accessibility:** Screen readers can better understand the layout and structure of the webpage, which is crucial for users with disabilities.
- **SEO Benefits:** Search engines give more weight to semantic tags, improving the search ranking of the page. For example, `<article>` or `<section>` tags make content more meaningful for search engines.
- **Code Clarity and Maintainability:** Semantic tags help developers understand the content structure without needing to rely on classes or IDs for clarification. This reduces errors and makes the code easier to maintain.
- **Examples of Semantic Tags:**
 - `<header>`: Defines the top section of the page, typically containing the logo and navigation.
 - `<footer>`: Denotes the footer section, often including copyright information or additional navigation.
 - `<nav>`: Used for navigation links.
 - `<article>`: Represents a piece of content, like a blog post or news article.
 - `<section>`: Used to group related content.

3.5 Additional Features

To enhance the functionality and user experience of the Attendance Tracker System, additional features can be incorporated:

1. **Search and Filter:** Users can search for attendance records by student name or ID. Filters can display records for a specific date range.
2. **Sorting:** Allow sorting of attendance data by student name, ID, or date for better organization.
3. **Pagination:** For large datasets, pagination can divide the attendance records into manageable chunks, improving page load times and user experience.
4. **Validation and Error Handling:** Use HTML5 form validation (like required fields, input type checks) to ensure data integrity when submitting attendance.
5. **Interactive User Interface:** Incorporating JavaScript to dynamically update the attendance table or provide feedback after submitting data (e.g., success or error messages).
6. **Responsive Design:** Ensure that the pages are mobile-friendly and accessible across various devices using CSS media queries.
7. **Export Functionality:** Allow users to export attendance data to CSV or Excel for offline analysis.

8. **Favicon Support:**HTML5 allows easy addition of a favicon using the <link rel="icon"> tag, giving a professional touch to the website.
9. **Lightweight Pages:**HTML5 structure helped keep the pages lightweight, leading to faster load times and better performance.
10. **Ease of Integration:** HTML5 pages easily integrated with CSS3 for styling and JavaScript for functionality without any compatibility issues.

3.6 Sample Code

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sample Page</title>
  </head>
  <body>
    <h1>Welcome to My Website</h1>
    <p>This is a paragraph of text. HTML makes it easy to structure content.</p>
    <a href="https://www.example.com">Visit Example</a>
  </body>
</html>
```


CHAPTER 4

CSS (Cascading style sheet)

4.1 What is CSS?

- CSS stands for Cascading Style Sheets.
- It is used to design and style HTML elements on a web page.
- CSS controls colors, fonts, layouts, spacing, borders, etc.
- It helps to separate the structure (HTML) from the design (CSS).
- Makes web pages look more attractive, organized, and professional.

4.2 Types of CSS

There are three types of CSS:

1. Inline CSS

- Written directly inside the HTML tag using the style attribute.
- Used for small and quick styling of single elements.

Example:

```
<p style="color:blue;">This is a blue paragraph.</p>
```

2. Internal CSS

- Written inside a <style> tag within the <head> section of the HTML page.
- Used when you want to style a single web page internally.

Example:

```
<head>
  <style>
    p {
      color: red;
    }
  </style>
</head>
```

3. External CSS

- Written in a separate file (with a .css extension).
- Linked to HTML using the <link> tag.
- Used to apply common styles to multiple pages.

Example:

In HTML:

```
<link rel="stylesheet" href="styles.css">
```

In styles.css:

```
p {  
    color: green;  
}
```

4.3 Role of CSS

CSS (Cascading Style Sheets) is a styling language that controls the look and feel of a web page. It separates content (HTML) from design (CSS).

- **Enhances Appearance:**
 - Defines colors, fonts, spacing, layout, and more.
 - Makes websites visually attractive and engaging.
- **Improves User Experience (UX):**
 - Proper use of CSS makes navigation easier and reading more comfortable.
 - Responsive design using CSS adapts websites to different screen sizes (phones, tablets, desktops).
- **Saves Time:**
 - A single CSS file can style multiple web pages.
 - Changes in one CSS file can instantly update the design across an entire site.
- **Separation of Concerns:**
 - HTML handles the structure/content.
 - CSS manages styling and presentation.
 - This makes code cleaner and easier to maintain.
- **Supports Consistency:**
 - Ensures that all pages of a website look consistent (same fonts, colors, button styles, etc.).

4.4 Internal vs External Styling

Feature	Internal CSS	External CSS
Where Used	Inside <style> tags in HTML <head>	In a separate .css file linked to HTML
When to Use	For small projects or single pages	For large websites or multiple pages
Advantage	Quick changes without extra files	Centralized control over site styling
Disadvantage	Harder to maintain with many styles	Needs extra HTTP request to load the file
Example	<style> p { color: red; } </style>	link rel="stylesheet" href="style.css"
Reusability	Not reusable across multiple pages	Easily reusable across pages

- **Internal CSS** is good for simple or one-page sites.
- **External CSS** is better for complex sites with many pages for easy maintenance and faster updates.

4.5 CSS Properties Used

Some important CSS properties often used in web development:

- **Color and Background:**
 - color: Text color.
Example: color: blue;
 - background-color: Background color of an element.
Example: background-color: yellow;
 - background-image: Sets a background image.
- **Text and Font:**
 - font-family: Changes the font style.
Example: font-family: Arial, sans-serif;
 - font-size: Controls the size of the text.
Example: font-size: 16px;
 - text-align: Aligns text (left, center, right, justify).
- **Box Model (Spacing):**
 - margin: Space outside the element.
 - padding: Space inside the element.
 - border: Adds borders around elements.
- **Layout:**
 - display: Defines how an element is displayed (block, inline, flex, grid).
 - position: Specifies the positioning method (static, relative, absolute, fixed).
 - flexbox and grid: Modern layout models for flexible designs.
- **Sizing:**
 - width and height: Sets dimensions of elements.
 - max-width, min-height: Set limits on sizing.
- **Effects:**
 - box-shadow: Adds shadow to elements.
 - opacity: Controls transparency.
- **Pseudo-classes and Pseudo-elements:**
 - :hover: Style an element when the mouse is over it. Example: button:hover { background-color: green; }
 - :after, ::before: Insert content before or after an element.

5.6 sample Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Attendance Tracker System</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
```

```
margin: 0;
padding: 0;
color: #333;
}
h1 {
text-align: center;
color: #4CAF50;
padding-top: 20px;
}
form {
width: 50%;
margin: 0 auto;
padding: 20px;
background-color: #fff;
border-radius: 8px;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}
fieldset {
border: 2px solid #4CAF50;
padding: 20px;
}
legend {
font-size: 1.5em;
font-weight: bold;
color: #4CAF50;
}
label {
font-size: 1.1em;
margin-bottom: 10px;
display: block;
}
input[type="text"] {
width: 100%;
padding: 10px;
margin: 8px 0;
border: 1px solid #ccc;
border-radius: 4px;
font-size: 1em;
}
input[type="submit"], input[type="reset"] {
background-color: #4CAF50;
color: white;
border: none;
padding: 10px 20px;
font-size: 1.1em;
border-radius: 4px;
cursor: pointer;
}
input[type="submit"]:hover, input[type="reset"]:hover {
```

```

        background-color: #45a049;
    }
    table {
        width: 80%;
        margin: 20px auto;
        border-collapse: collapse;
    }

    th, td {
        padding: 12px;
        text-align: left;
        border-bottom: 1px solid #ddd;
    }
    th {
        background-color: #4CAF50;
        color: white;
    }
    tr:hover {
        background-color: #f5f5f5;
    }
    hr {
        border: 0;
        border-top: 1px solid #ccc;
        margin: 20px 0;
    }
</style>
</head>
<body>

<h1>Attendance Tracker System</h1>

<form>
    <fieldset>
        <legend>Mark Attendance</legend>
        <label for="studentId">Student ID:</label><br>
        <input type="text" id="studentId" name="studentId" required><br><br>

        <label for="studentName">Student Name:</label><br>
        <input type="text" id="studentName" name="studentName" required><br><br>

        <input type="submit" value="Mark Present">
        <input type="reset" value="Clear">
    </fieldset>
</form>
<hr>
<h2>Attendance List</h2>
<table>
    <thead>
        <tr>

```

```
<th>Student ID</th>
<th>Student Name</th>
<th>Status</th>
</tr>
</thead>
<tbody>

</tbody>
</table>
</body>
</html>
```

Attendance Tracker System

Mark Attendance

Student ID:

Student Name:

Attendance List

Student ID	Student Name	Status
------------	--------------	--------

Figure : 5.1 Web page with both html and css

CHAPTER JAVA SCRIPT(JS)

5.1 What is JS (JavaScript)?

- JavaScript (JS) is a **high-level, interpreted** programming language mainly used to create **dynamic** and **interactive** content on websites.
- It runs **directly in the browser** (like Chrome, Firefox, Edge) without needing any external setup.
- JavaScript is a **client-side language** (runs on user's device) but can also be used **server-side** (like with Node.js).
- It is mainly used for tasks like **validating forms, creating animations, handling user events** (clicks, mouse moves), and **updating web pages without reloading** (AJAX).
- JS follows **object-oriented, event-driven, and functional** programming styles.

5.2 Functionality Added by JS

- **User Interaction:** Handle events like button clicks, mouse movements, keyboard input.
- **Dynamic Content:** Change HTML and CSS dynamically (e.g., show/hide elements, change colors, update text).
- **Form Validation:** Check user input before submitting data to the server (e.g., email validation).
- **Animations:** Create simple animations or use libraries (like GSAP) for complex animations.
- **Asynchronous Operations:** Load data in the background without refreshing the page (using AJAX, Fetch API).
- **Browser Control:** Access and control browser functions like history, cookies, local storage.
- **Real-Time Updates:** Power real-time apps (e.g., chat apps, live score updates) using WebSockets.
- **Server Communication:** Interact with servers and databases via APIs (using Fetch or Axios).

5.3 Form Validation Techniques

- **Client-Side Validation:** Ensures that form inputs are correct before sending data to the server, improving user experience and reducing server load.
- **HTML5 Built-in Validation:**
 - Uses attributes like required, minlength, maxlength, pattern, etc.
 - Examples:
 - type="email" validates email format.
 - required ensures the field is not left empty.
- **JavaScript Validation:**
 - Offers more control and customization over the validation process.

- Example: Checking if an input field contains only numbers or matches a regular expression pattern.
- **Regex (Regular Expressions):**
 - Used to validate complex patterns, such as email, phone number, or password formats.

Required Fields Check:

- Ensure fields like name, email, and password are filled out.

Pattern Matching:

- Ensure inputs follow a specified pattern, like validating a phone number or zip code.

On-the-Fly Validation:

- Using JavaScript events like onkeyup, onchange to validate fields while the user types.
- Example: Checking password strength as the user types.

5.4 Attendance Status Update Logic

- **Student Identification:** Identify each student uniquely, often by ID and/or name.

Example: Use a database or array where each student has a unique ID.

- **Track Attendance:** For each class, a student's attendance status can be **marked as "Present" or "Absent"**.

Use checkboxes or radio buttons in the UI to mark attendance.

Submit Attendance Data: When the user submits the form, the status (Present/Absent) for each student is recorded.

5.4 Elements of JavaScript

Variables:

- Used to store data values that can be referenced and manipulated.
- Types of variable declarations:
 - var: Traditional way, function-scoped (less commonly used today).
 - let: Block-scoped, can be updated.
 - const: Block-scoped, cannot be reassigned.

Data Types:

- JavaScript has both **primitive** and **non-primitive** data types.
 - Primitive Types:** string, number, boolean, null, undefined, symbol, bigint.
 - Non-Primitive Type:** object (includes arrays, functions, and more).

Functions:

- Blocks of reusable code designed to perform a specific task.
- Can be declared using function keyword or as arrow functions (`()=>{}`).

Events:

- JavaScript can react to events such as user clicks, keyboard input, or page load.
- Event handling is done via functions or event listeners.

Error Handling:

- JavaScript provides ways to catch and handle errors during runtime.
- `try`, `catch`, `finally` are used to handle errors gracefully.

ES6 Features (Modern JavaScript):

- Arrow Functions:** Shorter syntax for writing functions.
- Template Literals:** String interpolation with `${}`.
- Destructuring:** Extracting values from objects or arrays into variables.
- Promises:** Handle asynchronous operations.
-

DOM Manipulation:

- The **Document Object Model (DOM)** is a programming interface for web documents. JavaScript can interact with the DOM to change the content, structure, and style of web pages dynamically.

5.6 SAMPLE CODE

```
<script>
const form = document.getElementById('attendanceForm');
const attendanceList = document.getElementById('attendanceList');
form.addEventListener('submit', function(event) {
  event.preventDefault();
  const studentId = document.getElementById('studentId').value;
  const studentName = document.getElementById('studentName').value;
  const newRow = document.createElement('tr');
  const idCell = document.createElement('td');
  idCell.textContent = studentId;
  const nameCell = document.createElement('td');
  nameCell.textContent = studentName;
  const statusCell = document.createElement('td');
```

```

statusCell.textContent = 'Present';
newRow.appendChild(idCell);
newRow.appendChild(nameCell);
newRow.appendChild(statusCell);
attendanceList.appendChild(newRow);
form.reset();
});
</script>

```

Attendance Tracker System

Mark Attendance

Student ID:

Student Name:

Mark Present
Clear

Attendance List

Student ID	Student Name	Status
233	JOHN	Present
123	LEO	Present

FIGURE 5.1 WEB page with html,css,and java script

CHAPTER 6

IMPLEMENTATION

6.1 LOGIN PAGE

```
<!DOCTYPE html>
<html>
  <head>
    <title>Login - Attendance Tracker</title>
    <style>
      body {
        margin: 0;
        padding: 0;
        font-family: Arial, sans-serif;
        background: #f4f4f4;
        display: flex;
        justify-content: center;
        align-items: center;
        height: 100vh;
      }
      .login-container {
        background: white;
        padding: 30px 40px;
        border-radius: 10px;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        width: 100%;
        max-width: 400px;
        box-sizing: border-box;
      }
      .login-container h2 {
        text-align: center;
        margin-bottom: 20px;
      }
      .login-container label {
        display: block;
        margin: 10px 0 5px;
      }
      .login-container input {
        width: 100%;
        padding: 10px;
        margin-bottom: 15px;
        border-radius: 5px;
        border: 1px solid #ccc;
        box-sizing: border-box;
      }
```

```

}
.login-container button {
  width: 100%;
  padding: 10px;
  background-color: #4caf50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
}
.login-container button:hover {
  background-color: #45a049;
}
.reset-button {
  width: 100%;
  padding: 10px;
  background-color: #f44336;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
  margin-top: 10px;
}
.reset-button:hover {
  background-color: #e53935;
}
</style>
</head>
<body>
<div class="login-container">
  <h2>Login</h2>
  <form onsubmit="return validateLogin()">
    <label for="username">Username:</label>
    <input
      type="text"
      id="username"
      placeholder="Enter username"
      required
    />
    <label for="password">Password:</label>
    <input type="password" id="password" placeholder="Enter password" required >
    <button type="submit">Login</button>
    <button type="reset" class="reset-button">Reset</button>
  </form>
</div>
<script>
function validateLogin() {

```

```

const username = document.getElementById("username").value;
const password = document.getElementById("password").value;
const validUsername = "admin";
const validPassword = "admin123";
if (username === validUsername && password === validPassword) {
    window.location.href = "home.html";
    return false;
} else {
    alert("Invalid username or password!");
    return false;
}
}
</script>
</body>
</html>

```

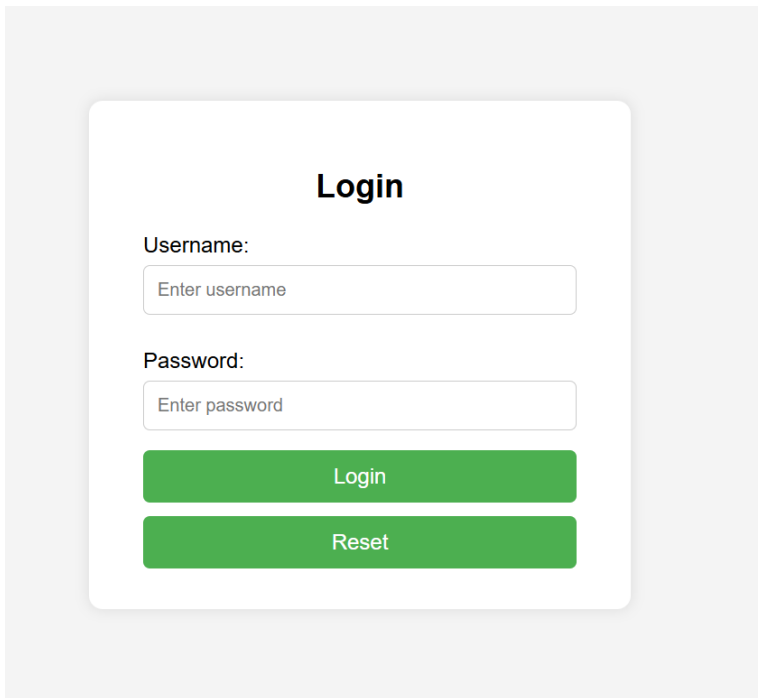


FIGURE 6.1 login page

6.2 HOME PAGE

```

<!DOCTYPE html>
<html>
  <head>
    <title>Dashboard - Attendance Tracker</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>

```

```
<h1 style="color: rgb(42, 21, 235); text-align: center">Select Subject</h1>
```

```
<div class="navbar">  
  <a href="attendance.html">Mark Attendance</a> |  
  <a href="report.html">Generate Report</a> |  
  <a href="students.html">Manage Students</a> |  
  <a href="login.html">Logout</a>  
</div>
```

```
<div class="dashboard-container">  
  <div class="card">  
    <h3>Select Subjects</h3>  
    <form id="subjects-form">  
      <label><input type="checkbox" /> English</label><br />  
      <label><input type="checkbox" /> Tamil</label><br />  
      <label><input type="checkbox" /> Graph Theory</label><br />  
      <label><input type="checkbox" /> Web Technology</label><br />  
      <label><input type="checkbox" /> Data Structure</label><br />  
      <label><input type="checkbox" /> FODS</label><br />  
    </form>  
    <div>  
      <h4>Selected Subjects:</h4>  
      <ul id="selected-subjects"></ul>  
    </div>
```

```
    <button id="start-attendance">Start Attendance</button>  
    <p id="error-message" style="color: red; display: none"></p>  
  </div>  
</div>
```

```
<script>  
function updateSelectedSubjects() {  
  const selectedSubjects = [];  
  const checkboxes = document.querySelectorAll(  
    'input[type="checkbox"]:checked'  
  );  
  checkboxes.forEach((checkbox) => {  
    selectedSubjects.push(checkbox.parentNode.textContent.trim());  
  });  
  
  const subjectsList = document.getElementById("selected-subjects");  
  subjectsList.innerHTML = "";  
  selectedSubjects.forEach((subject) => {  
    const li = document.createElement("li");  
    li.textContent = subject;  
    subjectsList.appendChild(li);  
  });  
}
```

```

document
.querySelectorAll('input[type="checkbox"]')
.forEach((checkbox) => {
  checkbox.addEventListener("change", updateSelectedSubjects);
});

document
.getElementById("start-attendance")
.addEventListener("click", function () {
  const selectedCheckboxes = document.querySelectorAll(
    'input[type="checkbox"]:checked'
  );
  const errorMessage = document.getElementById("error-message");

  if (selectedCheckboxes.length === 0) {
    errorMessage.textContent = "You didn't select any subject.";
    errorMessage.style.display = "block";
  } else {
    errorMessage.style.display = "none";

    const selectedSubjects = [];
    selectedCheckboxes.forEach((checkbox) => {
      selectedSubjects.push(checkbox.parentNode.textContent.trim());
    });

    // Store in localStorage
    localStorage.setItem(
      "selectedSubjects",
      JSON.stringify(selectedSubjects)
    );

    // Redirect
    window.location.href = "start.html";
  }
});
</script>
</body>
</html>

```

Select Subject

Mark Attendance | Generate Report
Logout | Manage Students |

Select Subjects

☐ English
☐ Tamil
☐ Graph Theory
☐ Web Technology
☐ Data Structure
☐ FODS

Selected Subjects:

Start Attendance

FIGURE 6.2 home page

6.3 STUDENTS PAGE

```
<!DOCTYPE html>
<html>
<head>
  <title>Manage Students</title>
  <link rel="stylesheet" href="style.css" />
  <style>
    .add-form {
      margin: 15px 0;
      display: none;
    }

    .add-form input {
      margin-right: 10px;
    }

    .add-form button {
      padding: 5px 10px;
    }
  </style>
</head>
<body>
  <header>
    <h1>Manage Students</h1>
    <button onclick="home()">Home</button>|
    <button onclick="logout()">Logout</button>
```



```
</header>

<section class="student-management">
  <h2>Student List</h2>
  <button onclick="toggleAddForm()">Add New Student</button>
  <button onclick="viewReport()">View Report</button>

  <!-- Add Student Form -->
  <div class="add-form" id="addForm">
    <input
      type="text"
      id="studentName"
      placeholder="Student Name"
      required
    />
    <input type="text" id="studentId" placeholder="Student ID" required />
    <button onclick="addStudent()">Add</button>
  </div>

  <!-- Student Table -->
  <table>
    <thead>
      <tr>
        <th>Student Name</th>
        <th>Student ID</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody id="studentTable">
      <tr>
        <td>John Doe</td>
        <td>12345</td>
        <td>
          <button onclick="editStudent(this)">Edit</button>
          <button onclick="deleteStudent(this)">Delete</button>
        </td>
      </tr>
      <tr>
        <td>Jane Smith</td>
        <td>67890</td>
        <td>
          <button onclick="editStudent(this)">Edit</button>
          <button onclick="deleteStudent(this)">Delete</button>
        </td>
      </tr>
    </tbody>
  </table>
</section>
```

```

<script>
function home() {
    alert("Returning to home...");
    window.location.href = "home.html";
}

function logout() {
    alert("Logging out...");
    window.location.href = "login.html";
}

function viewReport() {
    alert("Redirecting to report page");
    window.location.href = "report.html";
}

function toggleAddForm() {
    const form = document.getElementById("addForm");
    form.style.display = form.style.display === "none" ? "block" : "none";
}

function isDuplicate(name, id) {
    const rows = document.querySelectorAll("#studentTable tr");
    for (const row of rows) {
        const existingName = row.cells[0].innerText.trim().toLowerCase();
        const existingId = row.cells[1].innerText.trim();
        if (existingName === name.toLowerCase() || existingId === id) {
            return true;
        }
    }
    return false;
}

function addStudent() {
    const name = document.getElementById("studentName").value.trim();
    const id = document.getElementById("studentId").value.trim();

    if (name === "" || id === "") {
        alert("Please fill in both fields");
        return;
    }

    if (isDuplicate(name, id)) {
        alert("Student with this name or ID already exists!");
        return;
    }

    const table = document.getElementById("studentTable");
    const newRow = document.createElement("tr");

```

```

newRow.innerHTML = `
<td>${name}</td>
<td>${id}</td>
<td>
  <button onclick="editStudent(this)">Edit</button>
  <button onclick="deleteStudent(this)">Delete</button>
</td>
`;

table.appendChild(newRow);

document.getElementById("studentName").value = "";
document.getElementById("studentId").value = "";
document.getElementById("addForm").style.display = "none";
}

function deleteStudent(button) {
  const row = button.closest("tr");
  const name = row.cells[0].innerText;
  if (confirm(`Are you sure you want to delete ${name}?`)) {
    row.remove();
  }
}

function editStudent(button) {
  const row = button.closest("tr");
  const nameCell = row.cells[0];
  const idCell = row.cells[1];

  const currentName = nameCell.innerText;
  const currentId = idCell.innerText;

  const newName = prompt("Enter new name:", currentName)?.trim();
  const newId = prompt("Enter new student ID:", currentId)?.trim();

  if (newName && newId) {
    // Check for duplicates excluding the current row
    const rows = document.querySelectorAll("#studentTable tr");
    for (const r of rows) {
      if (r !== row) {
        const existingName = r.cells[0].innerText.trim().toLowerCase();
        const existingId = r.cells[1].innerText.trim();
        if (
          existingName === newName.toLowerCase() ||
          existingId === newId
        ) {
          alert("Another student with this name or ID already exists!");
          return;
        }
      }
    }
  }
}

```

```

    }
  }
}

nameCell.innerText = newName;
idCell.innerText = newId;
}
}
</script>
</body>
</html>

```

Manage Students		
Home Logout		
Student List		
Add New Student	View Report	
Student Name	Student ID	Actions
John Doe	12345	Edit Delete
Jane Smith	67890	Edit Delete

FIGURE 6.3 students page

6.4 SUBJECT PAGE

```

<!DOCTYPE html>
<html>
  <head>
    <title>Subject Attendance</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <h1>Subject Attendance</h1>
      <div>
        <button onclick="home()">Home</button>|
        <button onclick="logout()">Logout</button>
      </div>
    </header>

```

```
<h2>Selected Subjects</h2>
<ul id="subject-list"></ul>
<h3>Students</h3>
<table>
  <thead>
    <tr>
      <th>Student Name</th>
      <th>Percentage</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>John Doe</td>
      <td>90%</td>
    </tr>
    <tr>
      <td>Jane Smith</td>
      <td>85%</td>
    </tr>
  </tbody>
</table>
<script>
function home() {
  alert("Returning to home...");
  window.location.href = "home.html";
}
function logout() {
  alert("Logging out...");
  window.location.href = "login.html";
}
const subjects =
  JSON.parse(localStorage.getItem("selectedSubjects")) || [];
const list = document.getElementById("subject-list");
if (subjects.length === 0) {
  list.innerHTML = "<li>No subjects selected.</li>";
} else {
  subjects.forEach((subject) => {
    const li = document.createElement("li");
    li.textContent = subject;
    list.appendChild(li);
  });
}
</script>
</body>
</html>
```

Subject Attendance	
Home Logout	
Selected Subjects	
- Tamil - Graph Theory - Web Technology	
Students	
Student Name	Percentage
John Doe	90%
Jane Smith	85%

FIGURE 6.4 subject page

6.5 REPORT PAGE

```

<!DOCTYPE html>
<html>
  <head>
    <title>Attendance Reports</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <h1>Generate Reports</h1>
      <button onclick="home()">Home</button>|
      <button onclick="logout()">Logout</button>
    </header>
    <section class="reports">
      <h2>Attendance Report</h2>
      <form>
        <label for="start-date">Start Date:</label>
        <input type="date" id="start-date" name="start-date" />
        <label for="end-date">End Date:</label>
        <input type="date" id="end-date" name="end-date" />
        <button type="submit">Generate Report</button>
      </form>
      <h3>Generated Report</h3>
      <table>
        <thead>
          <tr>
            <th>Student Name</th>
            <th>Attendance Percentage</th>
          </tr>

```

```

</thead>
<tbody>
  <tr>
    <td>John Doe</td>
    <td>90%</td>
  </tr>
  <tr>
    <td>Jane Smith</td>
    <td>85%</td>
  </tr>
</tbody>
</table>
</section>
<script>
function home() {
  alert("Returning to home...");
  window.location.href = "home.html";
}
function logout() {
  alert("Logging out...");
  window.location.href = "login.html";
}
</script>
</body>
</html>

```

Generate Reports

[Home](#) | [Logout](#)

Attendance Report

Start Date:

End Date:

Generate Report

Generated Report

Student Name	Attendance Percentage
John Doe	90%
Jane Smith	85%

FIGURE 6.5 report page

ATTENDANCE PAGE

```
<!DOCTYPE html>
<html>
  <head>
    <title>Mark Attendance</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <h1>Mark Attendance</h1>
      <button onclick="home()">Home</button>|
      <button onclick="logout()">Logout</button>
    </header>

    <section class="attendance-form">
      <h2>Select Date and Class</h2>
      <form id="attendanceForm">
        <label for="date">Date:</label>
        <input type="date" id="date" name="date" required />

        <label for="class">Class:</label>
        <select id="class" name="class" required>
          <option value="Technical English">Technical English</option>
          <option value="Tamils and Technology">Tamils and Technology</option>
          <option value="Web Technology">Web Technology</option>
          <option value="DLC">DLC</option>
          <option value="Graph Theory">Graph Theory</option>
          <option value="Data Structure">Data structure</option>
        </select>
      <h3>Students</h3>
      <table>
        <thead>
          <tr>
            <th>Student Name</th>
            <th>Status</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>John Doe</td>
            <td>
              <select>
                <option value="Present">Present</option>
                <option value="Absent">Absent</option>
                <option value="Late">Late</option>
              </select>
            </td>
          </tr>
        </tbody>
      </table>
    </section>
  </body>
</html>
```



```

        <tr>
        <td>Jane Smith</td>
        <td>
            <select>
                <option value="Present">Present</option>
                <option value="Absent">Absent</option>
                <option value="Late">Late</option>
            </select>
        </td>
        </tr>
    </tbody>
</table>

    <button type="submit">Submit Attendance</button>
</form>
</section>
<script>
function home() {
    alert("Returning to home...");
    window.location.href = "home.html";
}

function logout() {
    alert("Logging out...");
    window.location.href = "login.html";
}
document
.getElementById("attendanceForm")
.addEventListner("submit", function (e) {
    e.preventDefault();
    alert("Attendance Submitted");
});
</script>
</body>
</html>

```

Mark Attendance

[Home](#) | [Logout](#)

Select Date and Class

Date:

dd / mm / yyyy

Class:

Technical English

Students	
Student Name	Status
John Doe	Present
Jane Smith	Present

Submit Attendance

FIGURE 6.6 attendance page

CHAPTER 7

CONCLUSION

The Attendance Tracker System, developed using front-end technologies like HTML, CSS, and JavaScript, provides an efficient and user-friendly solution for managing attendance digitally. The project showcases the potential of front-end development not only for building visually appealing interfaces but also for handling logical operations and dynamic data management entirely on the client side. Key features of the system include adding new students, marking their attendance as present or absent, updating records, and preventing duplicate entries unless both the name and ID match exactly. The design focuses on simplicity, responsiveness, and accessibility, ensuring a smooth experience across different devices such as desktops, tablets, and smartphones.

By using real-time updates through JavaScript, the system removes the need for backend servers or databases, making it lightweight and fast. Despite challenges like ensuring data consistency and maintaining dynamic updates without storage persistence, the system efficiently manages data during a session, highlighting the power of front-end logic and scripting. Future enhancements could include local storage integration for offline data saving, login systems for different user roles, attendance reports, and graphical data visualization to make the system even more comprehensive.

Overall, the Attendance Tracker System proves that even with just front-end technologies, it is possible to create practical, scalable, and effective solutions that address real-world needs. This project stands as a strong example of how thoughtful design, smart coding practices, and user-centric development can result in powerful applications without complex infrastructure.

Furthermore, the development of the Attendance Tracker System entirely with front-end technologies reinforces the concept that meaningful digital transformation does not always require complex or resource-intensive solutions. By emphasizing the role of the user experience (UX) and client-side operations, the project shows how real-time, interactive systems can be achieved efficiently with careful planning and logical structuring. This project also reflects an important learning journey — from understanding user requirements, designing intuitive workflows, implementing dynamic behaviors, to testing and refining for maximum usability. It highlights the importance of validation, error handling, and feedback mechanisms in enhancing system reliability and user trust. In a broader perspective, the system demonstrates how front-end-only applications can be highly valuable for small institutions, startups, or scenarios where backend integration is either not feasible or not immediately necessary. Additionally, the scalability of this system opens doors for future improvements where advanced features like role-based access (teacher, admin, student), attendance analytics dashboards, monthly attendance reports, and cloud synchronization could be seamlessly integrated.

REFERENCE

1. **"Database System Concepts"** – Abraham Silberschatz, Henry F. Korth, and S. Sudarshan
Used for: Understanding database design and management techniques, essential for storing, updating, and retrieving attendance data efficiently.
2. **"Web Development and Design Foundations with HTML5"** – Terry Felke-Morris
Used for: Building the front-end interface of the Attendance Tracker System using HTML5, CSS, and responsive design principles to ensure user-friendliness.
3. **"PHP and MySQL Web Development"** – Luke Welling and Laura Thomson
Used for: Learning server-side scripting and database integration for handling dynamic operations like student login, attendance marking, and report generation.
4. **"Modern Full-Stack Development: Using TypeScript, React, Node.js, Webpack, and Docker"** – Frank Zammetti
Used for: Providing full-stack development strategies that help in creating a complete, scalable attendance tracking web application.
5. **"Designing Interfaces: Patterns for Effective Interaction Design"** – Jenifer Tidwell
Used for: Improving the user experience (UX) and interface design of the system, ensuring that attendance marking and viewing are intuitive and efficient for users.
6. **"Fundamentals of Database Systems"** – Ramez Elmasri and Shamkant B. Navathe
Used for: Deepening knowledge of relational databases, normalization, and query optimization, crucial for managing large volumes of student attendance records efficiently.
7. **"The UX Book: Agile UX Design for a Quality User Experience"** – Rex Hartson and Pardha Pyla
Used for: Offering a detailed methodology for designing and testing the user experience, helping to create a seamless and accessible Attendance Tracker System for different users.
8. **"Building Web Apps with WordPress: WordPress as an Application Framework"** – Brian Messenlehner and Jason Coleman
Used for: Exploring how web platforms can be used to develop scalable applications